Ejercicios de aplicación 🤓

L3: Implementación de rn en Python

Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que, a continuación, encontrarás diferentes desafíos que podrás resolver de forma independiente y a tu ritmo.

Más adelante, conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

Diseñar, implementar y entrenar una red neuronal artificial en Python para resolver un problema de regresión simple utilizando Keras.

<u>Objetivo</u>
Crear un modelo secuencial con Keras que prediga un valor numérico (problema de regresión) a partir de un dataset sintético. Este desafío te permitirá poner en práctica conceptos como capas densas, funciones de activación, función de pérdida, optimizador, learning rate, epochs, y evaluación de resultados.

<u>Pasos sugeridos</u>

- *Generar o importar un conjunto de datos para regresión (puedes usar make_regression de sklearn.datasets).*
- *Preprocesar los datos y dividirlos en entrenamiento y prueba.*
- *Construir una red neuronal con, al menos, lo siguiente:*
    - *Capa de entrada con input_shape.*
    - *Una o dos capas ocultas con activación ReLU.*
    - *Capa de salida con activación lineal.*
- *Compilar el modelo utilizando lo siguiente:*

- *loss='mean_squared_error'*
- *optimizer='adam'*

- *Entrenar el modelo y visualizar la evolución del error.*
- *Evaluar el rendimiento en los datos de prueba.*

2. **¿Dónde se lleva a cabo?** 👩‍💻

   Visual Studio Code

3. **Tiempo de dedicación** ⏳

   2 horas

4. **Recursos** 🛠️

   Dataset: sklearn.datasets.make_regression()

   Documentación:

- https://keras.io/guides/sequential_model/
- *https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_regression.html*

5. **Plus** ➕

Modifica el modelo para probar distintos escenarios:

- *Cambia el número de capas y neuronas ocultas.*
- *Experimenta con diferentes tasas de aprendizaje (learning rate).*
- *Visualiza las predicciones reales vs. predichas con un gráfico de dispersión.*
- *Agrega regularización (Dropout, L2) para practicar prevención del sobreajuste.*

6. ⚠️**Condición**

   Esta práctica o ejercitación no requiere ser entregada o evaluada por el mentor. No obstante, puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

L3: Implementación de rn en Python

```python
# Paso 1: Importar librerías
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Paso 2: Crear dataset sintético
X, y = make_regression(n_samples=1000, n_features=1, noise=15)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Paso 3: Escalar datos
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Paso 4: Definir modelo
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(1,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))  # salida regresión

# Paso 5: Compilar modelo
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

# Paso 6: Entrenar
history = model.fit(X_train, y_train, epochs=50, validation_split=0.2)
```

```python
# Paso 7: Evaluar
mse, mae = model.evaluate(X_test, y_test)
print(f'MSE: {mse:.2f}, MAE: {mae:.2f}')

# Paso 8: Visualizar error de entrenamiento
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida durante el entrenamiento')
plt.xlabel('Épocas')
plt.ylabel('Error cuadrático medio')
plt.legend()
plt.show()
```